

Mastering SQL Joins with Bridge Tables

A Practical Guide for Working with Unfamiliar Data

Data Engineering Team

Enterprise Data Solutions

August 28, 2026

Agenda

- 1 Introduction to Complex Joins
- 2 Understanding Bridge Tables
- 3 Data Discovery Process
- 4 Multi-Table Join Strategies
- 5 Data Volume Handling
- 6 Python & PySpark Integration
- 7 Working with Limited Information
- 8 Performance Optimization
- 9 Real-World Example
- 10 Q&A

Introduction: The Challenge

Working with Unfamiliar Data

- Unknown table structures and relationships
- Inconsistent naming conventions
- Limited documentation
- Complex business logic
- Joining 6-7+ tables regularly

Key Objective

Develop a systematic approach to navigate these challenges using modern tools like Copilot.

Understanding Bridge Tables

What is a Bridge Table?

- Junction/Associative tables
- Facilitate many-to-many relationships
- Store relationship metadata

Common Example

- Orders ↔ Products
- Students ↔ Courses
- Patients ↔ Doctors

Table	Purpose
Product	Product info
Order	Order header
OrderProduct	Bridge table

Figure: Bridge Table Structure

Bridge Table SQL Example

```
1 CREATE TABLE OrderProduct (  
2   OrderID INT,  
3   ProductID INT,  
4   Quantity INT,  
5   UnitPrice DECIMAL(10,2),  
6   PRIMARY KEY (OrderID, ProductID),  
7   FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),  
8   FOREIGN KEY (ProductID) REFERENCES Products(ProductID)  
9 );  
10
```

Listing 1: Typical Bridge Table

- Composite primary key
- References both parent tables
- Contains relationship-specific attributes

Step 1: Data Discovery

Discovery Actions

- 1 Identify available tables
- 2 Examine sample data
- 3 Check data types
- 4 Review constraints

```
1  -- For Teradata
2  SELECT * FROM dbc.tables
3  WHERE databasename = 'YOUR_DB';
4
5  -- Check table structure
6  SHOW TABLE YOUR_TABLE;
7
8  -- Sample data exploration
9  SELECT * FROM YOUR_TABLE SAMPLE 100;
```

Listing 2: Discovery Queries

Join Path Identification

Guidelines for 6-7 Table Joins

- Start with fact tables
- Join dimension tables through bridge tables
- Use LEFT JOIN for optional relationships
- Document join logic carefully

Best Practices

- 1 Use meaningful aliases
- 2 Break queries with CTEs
- 3 Test incrementally
- 4 Validate results after each join

Multi-Table Join Example

```
1  WITH order_summary AS (  
2    SELECT  
3    o.OrderID,  
4    o.OrderDate,  
5    o.CustomerID,  
6    COUNT(op.ProductID) AS ProductCount  
7    FROM Orders o  
8    LEFT JOIN OrderProduct op  
9    ON o.OrderID = op.OrderID  
10   GROUP BY o.OrderID, o.OrderDate, o.CustomerID  
11  )  
12  SELECT  
13  c.CustomerName,  
14  os.OrderDate,  
15  p.ProductName,  
16  op.Quantity,  
17  p.Price * op.Quantity AS TotalAmount  
18  FROM order_summary os  
19  JOIN Customers c ON os.CustomerID = c.CustomerID  
20  JOIN OrderProduct op ON os.OrderID = op.OrderID  
21  JOIN Products p ON op.ProductID = p.ProductID
```


Data Volume Strategy

Volume	Tool	Strategy
100K-300K	Pandas	In-memory
300K-2M	PySpark	Distributed
>2M	Teradata	DB-side

Table: Processing Strategy by Data Volume

- Consider performance vs. flexibility
- Balance between SQL and Python
- Choose based on business requirements

Pandas Integration

```
1  import pandas as pd
2  import teradatadf
3
4  conn = teradatadf.connect(
5  host='your_host',
6  username='your_user',
7  password='your_pass'
8  )
9
10 query = """
11 SELECT * FROM large_table
12 WHERE conditions
13 """
14 df = pd.read_sql(query, conn)
15
16 # Further processing
17 result = df.groupby('category').agg({
18     'sales': 'sum',
19     'quantity': 'count'
20 })
```

PySpark for Larger Datasets

```
1  from pyspark.sql import SparkSession
2  import teradatasql
3
4  spark = SparkSession.builder \\\
5  .appName("Teradata_Integration") \\\
6  .getOrCreate()
7
8  df_spark = spark.read \\\
9  .format("jdbc") \\\
10 .option("url", "jdbc:teradata://host") \\\
11 .option("dbtable", "large_table") \\\
12 .option("user", "username") \\\
13 .option("password", "password") \\\
14 .load()
15
16 result_spark = df_spark.groupBy("category") \\\
17 .sum("sales") \\\
18 .filter("sum(sales) > 1000")
19
```

Listing 5: PySpark Integration

Handling Unknown Data

Exploratory Techniques

- Column profiling
- Relationship discovery
- Business context gathering
- Incremental testing

```
1  -- Check for nulls
2  SELECT
3      column_name ,
4      COUNT(*) AS total ,
5      SUM(CASE WHEN column_name IS NULL
6          THEN 1 ELSE 0 END) AS null_count
7  FROM your_table;
```

Listing 6: Defensive Querying

Performance Tips

Indexing

- Join column indexes
- Covering indexes
- Partition strategy

Query Optimization

- EXPLAIN PLAN
- Early filtering
- Avoid SELECT *
- UNION vs UNION ALL

Memory Consideration

- Monitor memory usage
- Use batch processing for large joins
- Consider temp tables

Real-World Example: Sales Analysis

Business Requirement

Analyze sales trends with customer demographics and product categories across 7 tables.

- 1 Data Discovery: Identify 7 tables
- 2 Join Planning: Map relationships
- 3 Query Development: Build incrementally
- 4 Performance Tuning: Optimize for volume

Complete Sales Analysis Query

```
1  WITH sales_data AS (  
2      SELECT s.SaleID, s.SaleDate,  
3      s.CustomerID, s.ProductID,  
4      s.Quantity, s.SaleAmount,  
5      sb.RegionID, sb.StoreID  
6  FROM Sales s  
7  JOIN SalesBridge sb ON s.SaleID = sb.SaleID  
8  WHERE s.SaleDate >= DATE '2024-01-01'  
9  ),  
10 customer_info AS (  
11     SELECT c.CustomerID, c.CustomerName,  
12     c.AgeGroup, c.IncomeLevel,  
13     r.RegionName  
14 FROM Customers c  
15 LEFT JOIN Regions r ON c.RegionID = r.RegionID  
16 )  
17 SELECT ci.RegionName, p.CategoryName,  
18 DATE_TRUNC('month', sd.SaleDate) AS SaleMonth,  
19 SUM(sd.SaleAmount) AS TotalRevenue,  
20 COUNT(DISTINCT sd.CustomerID) AS UniqueCustomers  
21 FROM sales_data sd
```

Key Takeaways

- 1 **Thorough Discovery** - Know your data before joining
- 2 **Strategic Planning** - Map relationships carefully
- 3 **Appropriate Tooling** - Choose based on data volume
- 4 **Incremental Development** - Build and test progressively
- 5 **Performance Awareness** - Optimize early and often

Modern Assistant

Leverage tools like Copilot for assistance while maintaining understanding of underlying data relationships and business context.

Summary

Complete Framework

- Systematic approach to complex joins
- Bridge table mastery
- Volume-appropriate strategies
- Python integration patterns
- Performance optimization

Success Factors

- Data understanding
- Technical flexibility
- Business context
- Continuous learning

- ① Teradata SQL Documentation
- ② Pandas Documentation - Data Processing
- ③ PySpark Documentation - Distributed Computing
- ④ SQL Performance Tuning Best Practices
- ⑤ Database Design Patterns for Enterprise Applications

Thank You!

Questions?